ONEDRAFT

CAD-AI — Aria Powered

Executable Application Stack & Runtime Architecture

Version 1.0

Status: Implementation Specification
Predecessor: Database Schema & OpenAPI Contract v0.1
Database: PostgreSQL
API: REST / JSON
API Version: `/api/v1`
Runtime: Python / TypeScript
Primary Model: Versioned OneDraft Project Model
AI Layer: Aria Powered
Execution Model: API + Domain Services + Workers + Event/Job Queue
Deployment Model: Containerized Services

---

1. IMPLEMENTATION PRINCIPLE

OneDraft shall now transition from architectural specification into an executable software system.

The implementation shall preserve the architectural distinction already established between:

Project Model

Aria Intelligence

Geometry Engine

Regulatory Engine

Validation Engine

Documentation Engine

Revision System

Customer Acceptance

No subsystem shall bypass the Project Model as the authoritative representation of the design.

The executable stack therefore becomes:

```
CLIENT
    ↓
```

```
WEB APPLICATION
    ↓
API GATEWAY
    ↓
APPLICATION SERVICES
    ↓
DOMAIN SERVICES
    ↓
PROJECT MODEL
    ↓
POSTGRESQL
```

with asynchronous computational systems operating alongside the transactional domain:

```
PROJECT MODEL
    ↓
JOB QUEUE
    ↓
WORKERS
    ├── Geometry Worker
    ├── Validation Worker
    ├── Drawing Worker
    ├── Document Worker
    └── AI Worker
```

---

2. TARGET REPOSITORY STRUCTURE

The executable OneDraft repository shall initially use:

```
OneDraft/
│
├── README.md
├── LICENSE
├── pyproject.toml
├── package.json
├── .env.example
├── docker-compose.yml
├── Makefile
│
├── apps/
│   ├── web/
│   ├── api/
│   └── worker/
│
├── packages/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   ├── geometry/
│   ├── compliance/
│   ├── documentation/
│   ├── aria/
│   └── contracts/
│
├── database/
```

```
│   ├── migrations/
│   ├── seeds/
│   └── fixtures/
│
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── openapi/
│   └── engineering/
│
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   └── e2e/
│
└── infrastructure/
    ├── docker/
    ├── deployment/
    └── monitoring/
```

This structure separates executable applications from reusable domain and infrastructure packages.

---

3. APPLICATIONS

The initial application layer shall contain three primary applications:

```
apps/web
apps/api
apps/worker
```

The web application is responsible for the user interface.

The API application is responsible for authenticated HTTP interaction.

The worker application is responsible for asynchronous computation.

---

4. WEB APPLICATION

The web application shall provide the primary OneDraft user environment.

The initial interface shall support:

```
Authentication
Organization selection
Project selection
Project dashboard
Project specification
Model browser
Level browser
Drawing browser
```

```
Drawing viewer
Redline interface
Validation results
Revision history
Document packages
Acceptance
```

The web client shall never communicate directly with PostgreSQL.

All persistent operations shall use the API contract.

---

5. WEB TECHNOLOGY

The initial frontend stack shall use:

```
TypeScript
React
Vite
Generated OpenAPI Client
```

The frontend shall consume the machine-readable OpenAPI contract.

Manual duplication of API request and response structures should be avoided.

---

6. FRONTEND STATE

Frontend state shall be divided into:

Server State

Project and model data obtained from the API.

UI State

Selections, panels, filters, active tools and temporary interface state.

Drawing State

Current drawing viewport, selected objects, redlines and annotation interactions.

Session State

Authenticated user, organization and project context.

The frontend must not become the authoritative source for project state.

---

7. API APPLICATION

The API application shall expose:

```
/api/v1
```

and implement the OpenAPI contract established in the previous specification.

The initial API framework shall use:

```
Python
FastAPI
Pydantic
SQLAlchemy
Alembic
```

The API shall be organized around application capabilities rather than database tables.

---

8. API LAYERING

The API application shall use:

```
HTTP Router
    ↓
Request Validation
    ↓
Authorization
    ↓
Application Service
    ↓
Domain Service
    ↓
Repository
    ↓
PostgreSQL
```

HTTP-specific concerns must not leak into the domain model.

---

9. DOMAIN PACKAGE

The domain package becomes the core of OneDraft.

Initial structure:

```
packages/domain/
├── project/
├── model/
├── building/
├── level/
├── space/
```

```
├── element/
├── geometry/
├── constraint/
├── revision/
├── drawing/
├── compliance/
├── validation/
├── document/
└── acceptance/
```

The domain package must remain independent of FastAPI and other HTTP infrastructure.

---

10. PROJECT MODEL

The Project Model is the authoritative computational representation of the project.

Conceptually:

```
Project
 ├── Requirements
 ├── Site
 ├── Buildings
 │    └── Levels
 │         ├── Spaces
 │         ├── Elements
 │         └── Relationships
 ├── Constraints
 ├── Assemblies
 └── Metadata
```

The model must be versionable.

Every meaningful mutation produces a traceable state transition.

---

11. DOMAIN IDENTIFIERS

All domain entities shall use UUID identifiers.

The domain layer shall not depend on database-generated integer identifiers.

Example:

```
ProjectId
BuildingId
LevelId
SpaceId
ElementId
RevisionId
ModelVersionId
CommandId
DrawingId
```

```
DocumentPackageId
```

These identifiers provide stable references throughout the platform.

---

12. DOMAIN ENUMS

The domain package shall define canonical enumerations for values such as:

```
ProjectStatus
ElementType
ConstraintType
CommandStatus
RevisionStatus
ValidationStatus
ValidationSeverity
DrawingType
DocumentStatus
AcceptanceStatus
JobStatus
```

Database and API representations shall derive from the domain vocabulary where practical.

---

13. PROJECT MODEL SERVICE

The first executable domain boundary shall be:

```
ProjectModelService
```

Its responsibility is to execute validated model operations.

Initial operations:

```
create_project
create_building
create_level
create_space
create_element
update_element
delete_element
create_constraint
create_relationship
execute_command
create_revision
restore_revision
```

No AI component directly modifies the database.

---

14. COMMAND PATTERN

Model modifications shall use explicit commands.

Conceptually:

```
Command
    ↓
Authorization
    ↓
Current Revision Verification
    ↓
Constraint Verification
    ↓
Domain Validation
    ↓
Mutation
    ↓
Change Record
    ↓
New Revision
    ↓
Audit Event
```

Commands are the controlled mutation interface of OneDraft.

---

15. COMMAND TYPES

Initial command types shall include:

```
CREATE_PROJECT
CREATE_BUILDING
CREATE_LEVEL
CREATE_SPACE
CREATE_ELEMENT
UPDATE_ELEMENT
MOVE_ELEMENT
ROTATE_ELEMENT
RESIZE_ELEMENT
DELETE_ELEMENT
CREATE_RELATIONSHIP
UPDATE_CONSTRAINT
GENERATE_VIEW
GENERATE_DRAWING
RUN_VALIDATION
CREATE_REDLINE
RESOLVE_REDLINE
GENERATE_DOCUMENT
```

The command vocabulary will expand as the domain matures.

---

16. OPTIMISTIC CONCURRENCY

Every mutation shall identify the revision against which the operation was requested.

Example:

```
expected_revision = 14
```

The server verifies that Revision 14 remains current.

If another operation has already produced Revision 15:

```
COMMAND REJECTED
```

The client must retrieve the current state before retrying.

This prevents silent overwriting of project changes.

---

17. TRANSACTION SERVICE

The Project Model Service shall execute mutations within database transactions.

Conceptually:

```
with transaction:
    authorize()
    verify_revision()
    load_model()
    validate_constraints()
    execute_command()
    record_changes()
    create_revision()
    record_event()
    commit()
```

A failure anywhere before commit causes rollback.

---

18. REPOSITORY LAYER

Database access shall be isolated behind repositories.

Initial repositories:

```
ProjectRepository
RequirementRepository
BuildingRepository
LevelRepository
SpaceRepository
ElementRepository
ConstraintRepository
```

```
RelationshipRepository
RevisionRepository
DrawingRepository
ValidationRepository
DocumentRepository
AuditRepository
```

Domain services must not contain raw SQL.

---

19. UNIT OF WORK

The infrastructure layer shall provide a Unit of Work abstraction.

Conceptually:

```
UnitOfWork
  ├── projects
  ├── requirements
  ├── buildings
  ├── levels
  ├── spaces
  ├── elements
  ├── constraints
  ├── revisions
  └── events
```

The Unit of Work controls transaction boundaries.

---

20. DATABASE IMPLEMENTATION

The initial persistence stack shall use:

```
PostgreSQL
SQLAlchemy
Alembic
```

Alembic migrations shall correspond to the database migration sequence established in the previous specification.

The first executable migration shall be:

```
001_initial_schema.sql
```

or its equivalent Alembic migration.

---

21. CONNECTION MANAGEMENT

The API shall use a managed PostgreSQL connection pool.

Configuration shall be supplied through environment variables.

Conceptually:

```
DATABASE_URL
DATABASE_POOL_SIZE
DATABASE_MAX_OVERFLOW
DATABASE_POOL_TIMEOUT
```

Credentials must never be committed to source control.

---

22. CONFIGURATION

Runtime configuration shall be centralized.

Initial configuration categories:

```
Database
API
Authentication
Queue
Object Storage
AI Provider
Geometry Engine
Compliance Sources
Document Engine
Observability
Billing
```

The application shall fail fast when required production configuration is missing.

---

23. ENVIRONMENT MODEL

Initial environments:

```
development
test
staging
production
```

Development configuration shall never be used automatically in production.

Environment-specific secrets shall remain outside source control.

---

24. WORKER APPLICATION

The worker application executes long-running jobs.

Initial worker categories:

```
AI Worker
Geometry Worker
Validation Worker
Drawing Worker
Document Worker
```

The worker process consumes jobs from the queue and reports status through the application infrastructure.

---

25. JOB QUEUE

OneDraft shall use an asynchronous job queue for computational work.

The initial implementation may use:

```
Redis
+
Celery
```

or an equivalent queue abstraction.

The application architecture must preserve the ability to replace the queue implementation later.

---

26. JOB LIFECYCLE

Every asynchronous operation follows:

```
REQUEST
 ↓
JOB CREATED
 ↓
QUEUED
 ↓
RUNNING
 ↓
VALIDATING
 ↓
COMPLETED
```

Failure states:

```
FAILED
CANCELLED
```

Jobs must be idempotent wherever practical.

---

27. JOB OWNERSHIP

Every job shall record:

```
job_id
project_id
job_type
requested_by
model_version_id
revision_id
status
progress
created_at
started_at
completed_at
error
result
```

The job must identify the exact project state against which it was executed.

---

28. STALE JOB PROTECTION

A worker must never blindly publish computational output.

Before finalizing a result it must verify:

```
project_id
model_version_id
revision_id
```

against the current state and the job's expected state.

If the model has advanced incompatibly, the job is marked stale or invalidated.

---

29. ARIA ORCHESTRATION

Aria shall operate through an application-level orchestration service.

Initial structure:

```
packages/aria/
├── orchestrator.py
├── planner.py
├── command_parser.py
├── constraint_context.py
```

```
├── proposal_engine.py
├── provider_adapter.py
└── schemas.py
```

Aria does not receive unrestricted infrastructure access.

---

30. ARIA EXECUTION PIPELINE

A natural-language request shall follow:

```
USER REQUEST
 ↓
ARIA PARSER
 ↓
PROJECT CONTEXT
 ↓
CONSTRAINT CONTEXT
 ↓
DESIGN INTENT
 ↓
COMMAND PROPOSAL
 ↓
DOMAIN VALIDATION
 ↓
COMMAND EXECUTION
 ↓
REVISION
 ↓
VALIDATION
 ↓
DRAWING UPDATE
```

Aria therefore produces controlled domain operations rather than arbitrary system mutations.

---

31. ARIA PROVIDER ADAPTER

The underlying model provider shall be abstracted.

Conceptually:

```
class AIProvider:
    def generate(self, request):
        ...
```

The provider adapter shall isolate:

```
model name
API credentials
token handling
provider-specific request formats
provider-specific response formats
```

```
retry behavior
rate limits
```

from the OneDraft domain.

---

32. ARIA CONTEXT

Aria requests shall receive structured project context.

Initial context:

```
Project
Requirements
Current Revision
Current Model Version
Affected Objects
Constraints
Code Manifest
Recent Validation Results
Relevant Drawings
Relevant Redlines
```

The complete database must not be blindly serialized into every AI request.

Context shall be selected according to task relevance.

---

33. ARIA OUTPUT CONTRACT

Aria must return structured intent.

Conceptually:

```
{
  "operation": "MOVE_ELEMENT",
  "targets": ["uuid"],
  "parameters": {
    "distance": 300,
    "unit": "mm",
    "direction": "NORTH"
  },
  "reason": "Move rear bedroom wall",
  "constraints_to_preserve": ["uuid"]
}
```

The application converts this into a domain command.

---

34. AI CONFIDENCE

AI-generated operations may include a confidence value.

Confidence shall not override deterministic domain validation.

The hierarchy remains:

```
AI INTENT
    ↓
DOMAIN RULES
    ↓
CONSTRAINTS
    ↓
VALIDATION
```

A high-confidence AI proposal that violates a hard constraint remains invalid.

---

35. DESIGN PROPOSALS

Where an operation is ambiguous or materially consequential, Aria shall produce a Design Proposal rather than directly execute a mutation.

The proposal contains:

```
Intent
Affected Objects
Proposed Changes
Rationale
Constraints
Estimated Impacts
```

The proposal may then be accepted, modified or rejected.

---

36. GEOMETRY ENGINE

The geometry subsystem becomes an independent computational package.

Initial structure:

```
packages/geometry/
├── primitives/
├── topology/
├── transforms/
├── intersections/
├── measurements/
├── bounding/
├── validation/
└── serialization/
```

The geometry engine is responsible for deterministic geometric computation.

---

37. GEOMETRY MODEL

The geometry engine shall support progressively:

```
Point
Line
Polyline
Arc
Circle
Curve
Polygon
Surface
Solid
Transform
Bounding Box
Plane
Section Plane
```

The exact implementation library may evolve without changing the Project Model API.

---

38. GEOMETRY REFERENCES

The relational database stores geometry references.

Large computational geometry payloads should be stored through the geometry/object-storage subsystem.

Every geometry artifact receives:

```
geometry_hash
geometry_version
model_version_id
storage_reference
bounding_box
```

---

39. GEOMETRY DETERMINISM

Given identical:

```
model_version
geometry_version
parameters
```

the geometry engine should produce materially identical results.

This is necessary for reproducible drawing generation and project reconstruction.

---

40. COMPLIANCE ENGINE

The compliance package shall implement jurisdiction-aware regulatory analysis.

Initial structure:

```
packages/compliance/
├── sources/
├── rules/
├── manifests/
├── applicability/
├── evaluators/
└── provenance/
```

The compliance engine consumes a Code Manifest.

---

41. COMPLIANCE PIPELINE

```
PROJECT JURISDICTION
 ↓
CODE SOURCES
 ↓
CODE MANIFEST
 ↓
APPLICABILITY ANALYSIS
 ↓
RULE SET
 ↓
PROJECT MODEL
 ↓
EVALUATION
 ↓
VALIDATION RESULTS
```

The regulatory layer must retain provenance for each material determination.

---

42. CODE SOURCE PROVENANCE

Each source shall identify:

```
authority
title
version
effective_date
retrieval_timestamp
```

```
content_hash
source_reference
```

A finalized revision shall retain the exact Code Manifest used for validation.

---

43. VALIDATION ENGINE

Validation shall operate independently of AI.

Initial validation categories:

```
CODE
SPATIAL
ACCESSIBILITY
FIRE
STRUCTURAL
COORDINATION
GEOMETRY
DOCUMENT
CONSISTENCY
```

The actual categories may expand by discipline.

---

44. VALIDATION RULE EXECUTION

A rule evaluator receives:

```
Project Model
Code Rule
Affected Objects
Relevant Parameters
```

and returns:

```
PASS
FAIL
WARNING
NOT_APPLICABLE
UNRESOLVED
```

Every result must identify its source.

---

45. VALIDATION AS A GATE

Certain downstream operations shall require successful validation.

For example:

```
FINAL DOCUMENT PACKAGE
```

may require:

```
No blocking validation failures
```

The system must distinguish:

```
BLOCKING
NON_BLOCKING
INFORMATIONAL
```

findings.

---

46. DOCUMENTATION ENGINE

The documentation package converts the Project Model into architectural documentation.

Initial structure:

```
packages/documentation/
├── views/
├── drawing/
├── sheets/
├── annotations/
├── dimensions/
├── schedules/
├── templates/
├── rendering/
└── export/
```

---

47. DOCUMENTATION PIPELINE

```
MODEL VERSION
 ↓
VIEW DEFINITIONS
 ↓
GEOMETRY EXTRACTION
 ↓
VIEW FILTERING
 ↓
ANNOTATION
 ↓
DIMENSIONS
 ↓
SCHEDULES
 ↓
SHEET COMPOSITION
```

```
  ↓
DOCUMENT EXPORT
```

The drawing is therefore generated from the model rather than authored as an isolated image.

---

48. VIEW GENERATION

Initial view types:

```
PLAN
REFLECTED CEILING PLAN
ROOF PLAN
ELEVATION
SECTION
DETAIL
3D
SITE PLAN
```

Each view references a specific Model Version.

---

49. ARCH D SHEETS

The initial sheet system shall support:

```
ARCH D
```

with configurable:

```
sheet dimensions
title block
project information
drawing numbers
drawing titles
revision block
scale
north arrow
graphic standards
notes
```

ARCH D is a sheet format, not a substitute for the underlying documentation model.

---

50. DRAWING GRAPHICS

The drawing renderer shall produce deterministic graphical output from:

```
Model
View
```

```
Visibility Rules
Annotation
Dimension
Sheet Layout
Graphic Standard
```

This permits a drawing to be regenerated when the model changes.

---

51. SCHEDULE ENGINE

Schedules shall derive from model data.

Initial schedule types:

```
Door Schedule
Window Schedule
Room Schedule
Finish Schedule
Equipment Schedule
Wall Type Schedule
Level Schedule
Area Schedule
```

Schedule values must be traceable to their source model objects.

---

52. REDLINE ENGINE

Redlines are treated as structured revision instructions rather than graphical marks alone.

A redline contains:

```
Drawing
Target Object
Markup Geometry
Instruction
Priority
Status
```

The system attempts to associate the redline with actual model objects.

---

53. REDLINE RESOLUTION

The resolution process becomes:

```
REDLINE
  ↓
INTERPRETATION
```

```
 ↓
TARGET IDENTIFICATION
 ↓
COMMAND PROPOSAL
 ↓
VALIDATION
 ↓
MODEL MUTATION
 ↓
NEW REVISION
 ↓
DRAWING REGENERATION
 ↓
REDLINE RESOLVED
```

A redline cannot become implemented merely because a drawing image changed.

---

54. OBJECT STORAGE

Large generated artifacts shall not be stored directly inside PostgreSQL.

Object storage shall contain:

```
Geometry Artifacts
Rendered Drawings
PDF Packages
Source Documents
Code Source Artifacts
Preview Images
Exports
```

PostgreSQL stores references and metadata.

---

55. OBJECT STORAGE INTERFACE

The infrastructure package shall expose an abstraction:

```
ObjectStorage
```

with operations:

```
put
get
delete
exists
presign
hash
```

The underlying provider can subsequently be changed without changing the domain.

---

56. FILE HASHING

Generated artifacts shall receive cryptographic hashes.

The initial implementation shall use:

```
SHA-256
```

Hashes shall be recorded for:

```
Code Sources
Geometry Artifacts
Document Packages
Exports
```

This supports provenance and reproducibility.

---

57. DOCUMENT PACKAGE

The document generation service shall assemble:

```
Drawing Set
Schedules
Notes
Reports
Validation Summary
Project Metadata
Revision Metadata
```

into the requested output format.

The package must reference the exact:

```
Project
Revision
Model Version
Code Manifest
Validation Run
```

from which it was generated.

---

58. ACCEPTANCE SERVICE

Customer acceptance becomes an explicit application capability.

The acceptance service verifies:

```
Project
Revision
```

```
Model Version
Document Package
Validation State
```

before recording acceptance.

Acceptance does not mutate the model.

It records the customer's decision regarding a specific project state.

---

59. AUDIT SERVICE

Every material operation generates an audit event.

Initial events:

```
PROJECT_CREATED
REQUIREMENT_CREATED
MODEL_CREATED
MODEL_CHANGED
COMMAND_PROPOSED
COMMAND_EXECUTED
REVISION_CREATED
REDLINE_CREATED
REDLINE_RESOLVED
VALIDATION_STARTED
VALIDATION_COMPLETED
DRAWINGS_GENERATED
DOCUMENT_GENERATED
ACCEPTANCE_RECORDED
```

Audit records remain append-only.

---

60. EVENT BUS

The application may publish domain events after successful transactions.

Initial events:

```
ProjectCreated
ModelChanged
RevisionCreated
ValidationCompleted
DrawingGenerated
DocumentPackageGenerated
AcceptanceRecorded
```

Events are not the authoritative project state.

The Project Model remains authoritative.

---

61. EVENT OUTBOX

To prevent loss of events between database commit and message publication, the implementation shall use an Outbox pattern.

Conceptually:

```
DATABASE TRANSACTION
    ↓
MODEL CHANGE
    +
OUTBOX EVENT
    ↓
COMMIT
    ↓
OUTBOX PUBLISHER
    ↓
MESSAGE QUEUE
```

This provides reliable event publication.

---

62. API IDEMPOTENCY

All externally initiated mutations shall support:

```
Idempotency-Key
```

The API shall associate the key with:

```
request_hash
response_hash
response_payload
```

A repeated request with the same key and identical request body shall return the original result.

A reused key with a different request body shall be rejected.

---

63. AUTHENTICATION SERVICE

The initial authentication architecture shall support:

```
User
Organization
Membership
Role
Scope
```

```
Session/Token
```

Authentication establishes identity.

Authorization establishes whether that identity can perform a requested operation.

---

64. AUTHORIZATION SERVICE

Every protected command shall evaluate:

```
Authenticated User
+
Organization Membership
+
Project Membership
+
Required Scope
+
Resource State
```

Authorization must occur before model mutation.

---

65. TENANCY ISOLATION

Organization and project boundaries must be enforced at the application layer.

A request authenticated for Organization A must not retrieve or mutate Project B belonging to Organization B.

Repository queries shall therefore include appropriate tenancy constraints.

---

66. SECURITY BOUNDARY

The system shall treat the following as privileged:

```
Database credentials
Object storage credentials
AI provider credentials
Queue credentials
Authentication signing keys
Production configuration
Billing configuration
```

These shall never be exposed to the frontend.

---

67. SECRET MANAGEMENT

Development may use:

```
.env
```

Production shall use an appropriate secret-management system.

The repository shall contain:

```
.env.example
```

but never real credentials.

---

68. OBSERVABILITY

The runtime shall implement:

```
Structured Logging
Metrics
Tracing
Health Checks
```

Every API request receives a:

```
request_id
```

Every asynchronous job receives a:

```
job_id
```

These identifiers must propagate through service calls.

---

69. LOGGING

Application logs shall use structured JSON in deployed environments.

Important fields include:

```
timestamp
level
service
request_id
job_id
project_id
revision_id
user_id
event
duration
```

```
error
```

Sensitive credentials and secrets must never be logged.

---

70. HEALTH ENDPOINTS

The API shall expose:

```
GET /health
GET /ready
```

Health indicates process availability.

Readiness indicates whether required dependencies are available.

---

71. SERVICE DEPENDENCIES

The initial runtime dependency graph is:

```
Web
  ↓
API
  ├── PostgreSQL
  ├── Redis / Queue
  └── Object Storage

Worker
  ├── PostgreSQL
  ├── Redis / Queue
  ├── Object Storage
  ├── Geometry Engine
  ├── Compliance Engine
  ├── Documentation Engine
  └── Aria Provider Adapter
```

---

72. LOCAL DEVELOPMENT

The initial local development environment shall be executable through:

```
docker-compose.yml
```

The minimum local services shall be:

```
postgres
redis
api
worker
```

```
web
```

Object storage may initially use a local-compatible implementation.

---

73. DEVELOPMENT COMMANDS

The repository should provide a common command interface.

Initial commands:

```
make install
make dev
make test
make lint
make format
make migrate
make seed
make openapi
make build
```

Equivalent package-manager commands may also be provided.

---

74. DATABASE INITIALIZATION

A new development environment shall be capable of:

```
Start PostgreSQL
 ↓
Run migrations
 ↓
Create development seed data
 ↓
Start API
 ↓
Start workers
 ↓
Start frontend
```

without manually modifying database tables.

---

75. OPENAPI GENERATION

The OpenAPI specification shall remain synchronized with the API implementation.

The repository shall provide:

```
/docs/openapi/openapi.yaml
```

and a mechanism to validate the implementation against it.

The generated frontend client shall derive from this contract.

---

76. PYDANTIC CONTRACTS

The API layer shall use Pydantic models for:

```
Request Models
Response Models
Command Models
Job Models
Validation Models
Document Models
```

The domain model remains conceptually separate from transport DTOs.

---

77. DOMAIN / API DTO SEPARATION

The system shall distinguish:

```
Domain Entity
```

from:

```
API Request
API Response
```

This prevents external API design decisions from becoming irreversible domain constraints.

---

78. TESTING STRATEGY

Testing shall occur at four levels:

```
Unit
Integration
Contract
End-to-End
```

Each level serves a different purpose.

---

79. UNIT TESTS

Unit tests shall cover:

```
Domain Rules
Constraint Evaluation
Command Validation
Geometry Primitives
Revision Logic
Permission Rules
Code Rule Evaluation
Drawing Calculations
```

Unit tests must execute without requiring the complete production stack where practical.

---

80. INTEGRATION TESTS

Integration tests shall verify:

```
API
PostgreSQL
Queue
Object Storage
Workers
Domain Services
```

The first database integration test must verify that a complete revision transaction can be committed and reconstructed.

---

81. CONTRACT TESTS

Contract tests shall verify that:

```
API implementation
```

conforms to:

```
openapi.yaml
```

The generated TypeScript client must be compatible with the API responses.

---

82. END-TO-END TEST

The first complete E2E test is the workflow defined by the previous specification:

```
CREATE ORGANIZATION
 ↓
CREATE USER
 ↓
CREATE PROJECT
 ↓
CREATE REQUIREMENTS
 ↓
CREATE BUILDING
 ↓
CREATE LEVELS
 ↓
CREATE SPACES
 ↓
CREATE WALLS
 ↓
CREATE DOORS
 ↓
CREATE WINDOWS
 ↓
GENERATE DRAWINGS
 ↓
SUBMIT ARIA MODIFICATION
 ↓
VALIDATE
 ↓
CREATE REVISION
 ↓
CREATE REDLINE
 ↓
RESOLVE REDLINE
 ↓
RUN VALIDATION
 ↓
GENERATE DOCUMENT PACKAGE
 ↓
HASH PACKAGE
 ↓
RECORD ACCEPTANCE
 ↓
RETRIEVE HISTORY
```

This becomes the primary integration/regression path.

---

83. FAILURE HANDLING

Failures must be explicit.

The system shall distinguish:

```
Validation Failure
Domain Conflict
Authorization Failure
Dependency Failure
AI Provider Failure
```

```
Geometry Failure
Document Generation Failure
Storage Failure
Database Failure
Timeout
Cancellation
Stale Revision
```

Errors shall not be silently converted into successful state.

---

84. RETRY POLICY

Retryable failures may include:

```
Temporary database connection failure
Temporary queue failure
Temporary object storage failure
AI provider timeout
Transient worker failure
```

Non-retryable failures include:

```
Constraint violation
Authorization failure
Invalid command
Invalid revision
Malformed request
Permanent validation failure
```

Retries must respect idempotency.

---

85. CANCELLATION

Long-running jobs shall support cancellation where technically practical.

A cancelled job must not publish partial output as current project state.

Workers must periodically check cancellation status during long operations.

---

86. PROJECT STATE LOCKING

OneDraft shall not require a global project lock for ordinary operations.

Concurrency shall primarily be managed through:

```
Revision Numbers
Optimistic Concurrency
```

```
Atomic Transactions
Job Version Checks
```

Temporary locks may be introduced for operations that genuinely require exclusive computation.

---

87. CACHE STRATEGY

Caching shall not become a second source of truth.

The initial cache may contain:

```
Frequently requested project metadata
Read-heavy lookup data
Code source metadata
Generated previews
```

Any cache can be invalidated and reconstructed from authoritative state.

---

88. SEARCH

The initial MVP shall use PostgreSQL search where sufficient.

Future search infrastructure may support:

```
Full-text project search
Element search
Document search
Semantic search
AI retrieval
```

Search indexes remain derived state.

---

89. AI RETRIEVAL

Future retrieval systems may index:

```
Project Requirements
Code Rules
Project Notes
Revisions
Validation Results
Drawing Metadata
Redlines
```

Retrieved information must retain provenance.

AI retrieval must never erase the distinction between source information and model-generated interpretation.

---

90. DOCUMENT IMPORT

The application shall eventually support source document ingestion.

Potential inputs:

```
PDF
Images
CAD files
Specifications
Survey documents
Code documents
Existing schedules
```

Imported documents become source artifacts.

They do not automatically become authoritative model state without interpretation and validation.

---

91. DOCUMENT INGESTION PIPELINE

```
UPLOAD
 ↓
OBJECT STORAGE
 ↓
HASH
 ↓
METADATA
 ↓
EXTRACTION
 ↓
CLASSIFICATION
 ↓
ARIA INTERPRETATION
 ↓
PROPOSED MODEL CHANGES
 ↓
VALIDATION
 ↓
USER ACCEPTANCE
```

This keeps imported information separate from confirmed project state.

---

92. CAD INTEROPERABILITY

The architecture shall reserve interfaces for:

```
DXF
DWG
IFC
SVG
PDF
STEP
OBJ
glTF
```

Support shall be introduced incrementally.

No interoperability format shall become the canonical Project Model.

---

93. CANONICAL MODEL

The canonical OneDraft representation remains:

```
OneDraft Project Model
```

External CAD/BIM formats are import/export representations.

This preserves the ability to evolve the internal computational model independently.

---

94. GEOMETRY / BIM SEPARATION

OneDraft shall distinguish:

```
Semantic Building Model
```

from:

```
Geometric Representation
```

An element may possess both:

```
semantic properties
```

and:

```
geometry references
```

This distinction is required for robust BIM/CAD interoperability.

---

95. DOCUMENT PROVENANCE

Every generated drawing shall be traceable through:

```
Drawing
 ↓
View
 ↓
Model Version
 ↓
Revision
 ↓
Project
```

Every document package shall additionally identify:

```
Code Manifest
Validation Run
```

This forms the document provenance chain.

---

96. RECONSTRUCTION ENGINE

OneDraft shall provide an internal reconstruction capability.

Given:

```
project_id
revision_id
model_version_id
```

the system reconstructs the corresponding model state.

Given:

```
project_id
revision_id
model_version_id
code_manifest_id
```

the system shall identify the regulatory and validation context.

---

97. HISTORICAL REPLAY

The revision system should eventually support:

```
Revision 1
 ↓
Revision 2
```

```
 ↓
Revision 3
 ↓
Revision 4
```

with change records capable of explaining the transition between each state.

Historical state must remain immutable.

---

98. MODEL DIFF

The platform shall eventually expose:

```
Model Diff
```

which identifies:

```
Added Objects
Removed Objects
Modified Objects
Moved Objects
Changed Parameters
Changed Relationships
Changed Constraints
```

This becomes the computational basis for redline impact analysis.

---

99. DRAWING IMPACT ANALYSIS

When an element changes, the system should determine which documentation is affected.

Conceptually:

```
Changed Element
 ↓
Dependency Graph
 ↓
Affected Views
 ↓
Affected Drawings
 ↓
Affected Sheets
 ↓
Affected Schedules
```

Only affected derived artifacts need regeneration where safe.

---

100. INCREMENTAL REGENERATION

The documentation engine shall support incremental regeneration.

For example:

```
Wall Changed
  ↓
Affected Plan
Affected Elevation
Affected Section
Affected Door/Window Schedule
```

Unrelated documentation should not be regenerated unnecessarily.

---

101. COMPUTATIONAL DEPENDENCY GRAPH

The relationship system becomes increasingly important as the model grows.

The graph connects:

```
Elements
Spaces
Levels
Assemblies
Constraints
Views
Drawings
Schedules
Validation Rules
Redlines
```

This graph supports impact analysis and efficient regeneration.

---

102. PERFORMANCE TARGET

The MVP shall prioritize correctness and determinism over premature optimization.

Initial performance objectives include:

```
Simple CRUD operations: sub-second under normal development conditions
Command validation: near-real-time for ordinary operations
Heavy geometry: asynchronous
Validation suites: asynchronous
Drawing generation: asynchronous
Document generation: asynchronous
```

Exact production SLAs will be established after profiling.

---

103. SCALING MODEL

The architecture shall permit independent scaling of:

```
API instances
Worker instances
Geometry workers
Validation workers
Drawing workers
Document workers
AI workers
```

The API must not be coupled to a single worker process.

---

104. MULTI-TENANCY SCALING

Organization and project boundaries permit horizontal scaling.

Future deployments may partition workloads by:

```
organization
project
region
worker pool
```

without changing the fundamental Project Model.

---

105. BILLING BOUNDARY

Billing shall remain outside the core Project Model.

The billing layer may record:

```
Subscription
Plan
Usage
Credits
Invoices
Entitlements
```

but billing state must not become a dependency of core model integrity.

The authorization layer may consult entitlements before initiating metered operations.

---

106. USAGE METERING

Future metered operations may include:

```
AI Commands
Geometry Computation
Validation Runs
Drawing Generation
Document Generation
Storage
Exports
```

Usage records should be generated from completed operations rather than client assertions.

---

107. FEATURE FLAGS

Feature flags may control incomplete capabilities.

Initial categories:

```
Experimental
Beta
Production
Deprecated
```

Feature flags shall not alter historical project state.

---

108. API RATE LIMITING

The API gateway shall eventually enforce rate limits based on:

```
User
Organization
IP
Endpoint
Operation Type
Plan Entitlement
```

Heavy computational endpoints shall have separate controls from ordinary CRUD endpoints.

---

109. AI RATE LIMITING

AI requests shall be independently controlled.

The system shall account for:

```
request frequency
token consumption
provider quotas
organization entitlements
job concurrency
```

A rate-limited AI provider must not cause corruption of the Project Model.

---

110. SECURITY TESTING

The implementation must eventually test:

```
Authentication bypass
Authorization bypass
Cross-tenant access
IDOR/resource enumeration
SQL injection
Malformed payloads
Replay attacks
Idempotency abuse
Job injection
File upload abuse
Object storage access
AI prompt injection
```

AI-originated instructions are untrusted input until translated into validated domain commands.

---

111. AI PROMPT-INJECTION BOUNDARY

Imported documents, project text, code-source text and external content must be treated as data.

Instructions contained inside retrieved content must not automatically become system instructions.

The AI orchestration layer shall preserve the hierarchy:

```
SYSTEM POLICY
 ↓
APPLICATION RULES
 ↓
DOMAIN CONSTRAINTS
 ↓
AUTHORIZED USER INTENT
 ↓
RETRIEVED DATA
```

Retrieved content cannot override higher-level controls.

---

112. FILE SECURITY

Uploaded files shall receive:

```
content hash
MIME verification
size limits
metadata
storage isolation
access controls
```

Files shall not automatically be executed.

---

113. BACKGROUND WORKER SECURITY

Workers shall operate with the minimum permissions necessary.

For example:

```
Drawing Worker
```

does not require unrestricted billing or identity permissions.

Service-to-service credentials shall be scoped.

---

114. DEPLOYMENT

The initial deployment model shall use containers.

Each service shall have an independently buildable image.

Initial services:

```
web
api
worker
```

Infrastructure services:

```
postgres
redis
object-storage
```

---

115. CONTAINERIZATION

Each application shall provide a production Dockerfile.

The development environment shall provide:

```
docker-compose.yml
```

Production orchestration may subsequently use:

```
Kubernetes
```

or another container orchestration platform.

The application must not depend on Kubernetes-specific behavior at the domain level.

---

116. CI PIPELINE

Every repository change should pass:

```
Formatting
Linting
Type Checking
Unit Tests
Integration Tests
OpenAPI Validation
Build
```

before being eligible for deployment.

---

117. MIGRATION SAFETY

Database migrations must be:

```
ordered
reviewable
repeatable
forward-compatible where practical
```

Applied migrations shall not be rewritten.

A corrective migration is created for every post-deployment schema correction.

---

118. BACKUP

PostgreSQL production data shall be backed up.

The recovery strategy shall eventually define:

```
Backup frequency
Retention
Point-in-time recovery
Restore testing
Disaster recovery
Recovery Point Objective
Recovery Time Objective
```

Backups must themselves be tested through restoration procedures.

---

119. DISASTER RECOVERY

The platform shall be capable of reconstructing the operational system from:

```
Database Backup
Object Storage
Source Repository
Configuration
Secrets
Migration History
```

The Project Model and its document provenance must remain recoverable.

---

120. DATA RETENTION

Retention policies shall distinguish:

```
Current project state
Historical revisions
Audit events
Source documents
Generated documents
Temporary jobs
Caches
```

Temporary computational artifacts may have shorter retention than authoritative project records.

---

121. DELETION MODEL

Normal user-facing deletion shall use soft deletion where appropriate.

Historical revision records remain immutable.

Physical deletion of data must respect:

```
legal retention requirements
organizational policy
billing requirements
audit requirements
customer agreements
```

---

122. API RESOURCE IMPLEMENTATION ORDER

The initial API implementation order shall be:

```
1. Authentication Context
2. Organizations
3. Projects
4. Requirements
5. Buildings
6. Levels
7. Spaces
8. Elements
9. Constraints
10. Revisions
11. Commands
12. Views
13. Drawings
14. Redlines
15. Compliance
16. Validation
17. Jobs
18. Documents
19. Acceptance
20. Events
```

This order follows the dependency structure of the domain.

---

123. FIRST EXECUTABLE SERVICE

The first service to become fully operational shall be:

```
ProjectModelService
```

It must support:

```
Create Project
Create Building
Create Level
```

```
Create Space
Create Element
Modify Element
Create Constraint
Create Revision
```

before Aria is granted mutation capability.

---

124. FIRST ARIA INTEGRATION

Once the ProjectModelService is operational, the first Aria integration shall implement one deterministic operation:

```
MOVE_ELEMENT
```

Example:

```
Move rear bedroom wall 300 mm north.
```

The system must:

```
interpret request
identify target
construct command
verify constraints
execute command
create revision
record change
record event
```

---

125. FIRST GEOMETRY INTEGRATION

The first geometry integration shall calculate the resulting position of the moved element.

The workflow becomes:

```
Revision 1
 ↓
MOVE_ELEMENT
 ↓
Revision 2
 ↓
Geometry Job
 ↓
Geometry Version 2
 ↓
Affected View
 ↓
Updated Drawing
```

---

126. FIRST VALIDATION INTEGRATION

The first validation integration shall verify a simple deterministic rule.

Example:

```
minimum room dimension
```

The test shall demonstrate:

```
Model
 ↓
Rule
 ↓
Evaluation
 ↓
PASS / FAIL
 ↓
Validation Result
```

before complex jurisdictional rule systems are introduced.

---

127. FIRST DRAWING INTEGRATION

The first drawing renderer shall generate a simple:

```
PLAN
```

from:

```
Building
Level
Spaces
Walls
Doors
Windows
```

The resulting drawing shall be associated with:

```
Model Version
Revision
View
Sheet
```

---

128. FIRST DOCUMENT INTEGRATION

The first document package shall generate:

```
ARCH D PDF
```

containing at minimum:

```
Title Block
One Plan
One Elevation
One Section
Revision Information
```

The package receives a SHA-256 hash.

---

129. FIRST COMPLETE SYSTEM LOOP

The first complete executable loop is therefore:

```
USER
 ↓
WEB
 ↓
API
 ↓
PROJECT MODEL
 ↓
POSTGRESQL
 ↓
ARIA
 ↓
COMMAND
 ↓
REVISION
 ↓
GEOMETRY
 ↓
VALIDATION
 ↓
DRAWING
 ↓
DOCUMENT
 ↓
ACCEPTANCE
```

This is the first genuine OneDraft system.

---

130. IMPLEMENTATION ARTIFACTS

The next repository artifacts shall be created in this order:

`database/migrations/001_initial_schema.sql`

`docs/openapi/openapi.yaml`

`packages/domain/project/model.py`

`packages/domain/model/entities.py`

`packages/domain/model/commands.py`

`packages/application/project_model_service.py`

`packages/infrastructure/database.py`

`packages/infrastructure/repositories.py`

`apps/api/main.py`

`apps/api/routes/projects.py`

`apps/api/routes/model.py`

`apps/api/routes/commands.py`

`apps/worker/main.py`

`packages/geometry/engine.py`

`packages/compliance/engine.py`

`packages/documentation/engine.py`

`packages/aria/orchestrator.py`

`tests/e2e/test_first_project_loop.py`

---

131. IMPLEMENTATION SEQUENCE

The actual build sequence shall be:

```
DATABASE
 ↓
DOMAIN TYPES
 ↓
REPOSITORIES
 ↓
UNIT OF WORK
 ↓
PROJECT MODEL SERVICE
 ↓
```

```
API
 ↓
OPENAPI CLIENT
 ↓
WORKER QUEUE
 ↓
ARIA COMMAND PIPELINE
 ↓
GEOMETRY
 ↓
VALIDATION
 ↓
DRAWING
 ↓
DOCUMENT
 ↓
WEB INTERFACE
 ↓
END-TO-END TEST
```

The system should not attempt to build the complete CAD renderer before the domain transaction loop works.

---

132. FIRST VERTICAL SLICE

The first production-shaped vertical slice shall be deliberately narrow.

It shall perform:

```
Create Project
 ↓
Create Building
 ↓
Create Level
 ↓
Create Space
 ↓
Create Wall
 ↓
Move Wall
 ↓
Create Revision
 ↓
Validate
 ↓
Generate Plan
```

If this vertical slice succeeds, the architecture has demonstrated that the core OneDraft computational loop works.

---

133. DEFINITION OF DONE

The first implementation milestone is complete when a clean environment can execute:

```
docker compose up
```

then:

```
POST /api/v1/projects
```

to create a project,

followed by authenticated API operations that create model objects and modify them through the Project Model Service.

The resulting project must be persisted in PostgreSQL and reconstructable from its revision history.

---

134. SECOND DEFINITION OF DONE

The second milestone is complete when:

```
Aria
```

can receive:

```
Move the rear bedroom wall 300 mm north.
```

and produce a validated domain command that results in:

```
Revision N+1
```

without direct AI database access.

---

135. THIRD DEFINITION OF DONE

The third milestone is complete when the modified model automatically produces:

```
updated geometry
updated plan
updated elevation
updated section
updated schedules
```

where affected documentation can be determined through the dependency graph.

---

136. FOURTH DEFINITION OF DONE

The fourth milestone is complete when OneDraft can generate:

```
ARCH D document package
```

whose contents are traceable to:

```
Project
Model Version
Revision
Code Manifest
Validation Run
```

and whose final artifact has a cryptographic hash.

---

137. FIFTH DEFINITION OF DONE

The fifth milestone is complete when a customer can:

```
review
redline
request revision
receive regenerated documentation
review validation
accept
```

without breaking the project provenance chain.

---

138. ENGINEERING PRINCIPLE

The OneDraft stack shall be built from the inside outward.

The order is:

```
TRUTH
 ↓
MODEL
 ↓
COMMAND
 ↓
REVISION
 ↓
COMPUTATION
 ↓
DOCUMENTATION
 ↓
INTERFACE
```

The user interface is therefore the visible surface of a much deeper computational system.

---

139. ARIA'S POSITION IN THE STACK

Aria is not the database.

Aria is not the Project Model.

Aria is not the geometry engine.

Aria is not the regulatory authority.

Aria is the intelligence and orchestration layer that interprets authorized project intent and translates it into controlled computational operations.

The architecture therefore remains:

```
USER INTENT
      ↓
     ARIA
      ↓
DOMAIN COMMAND
      ↓
PROJECT MODEL
      ↓
DETERMINISTIC SYSTEMS
      ↓
DOCUMENTATION
```

---

140. FUNDAMENTAL SYSTEM BOUNDARY

The most important implementation rule remains:

```
AI MAY PROPOSE.
DOMAIN SERVICES DECIDE.
DATABASE PERSISTS.
WORKERS COMPUTE.
VALIDATION VERIFIES.
DOCUMENTATION REPRESENTS.
CUSTOMER ACCEPTS.
```

This boundary prevents OneDraft from becoming an uncontrolled generative drawing system.

---

141. STACK STATUS

ONEDRAFT EXECUTABLE APPLICATION STACK v0.1

STATUS: ESTABLISHED

The OneDraft architecture has now progressed from:

```
PRODUCT DEFINITION
 ↓
SYSTEM ARCHITECTURE
 ↓
TECHNOLOGY ARCHITECTURE
 ↓
DOMAIN MODEL
 ↓
DATABASE SCHEMA
 ↓
OPENAPI CONTRACT
 ↓
EXECUTABLE APPLICATION STACK
```

The architecture is now sufficiently defined to begin implementation.

---

142. NEXT IMPLEMENTATION STAGE

The next stage is no longer another conceptual architecture document.

The next stage is executable code.

The first implementation package shall create:

```
001_initial_schema.sql
```

then:

```
openapi.yaml
```

then:

```
domain_types.py
```

then:

```
project_model_service.py
```

followed by:

```
API
+
repositories
+
unit of work
+
worker queue
+
first geometry operation
+
```

```
first validation rule
+
first drawing renderer
```

The first objective is not to build every OneDraft feature simultaneously.

The objective is to make the following loop execute successfully:

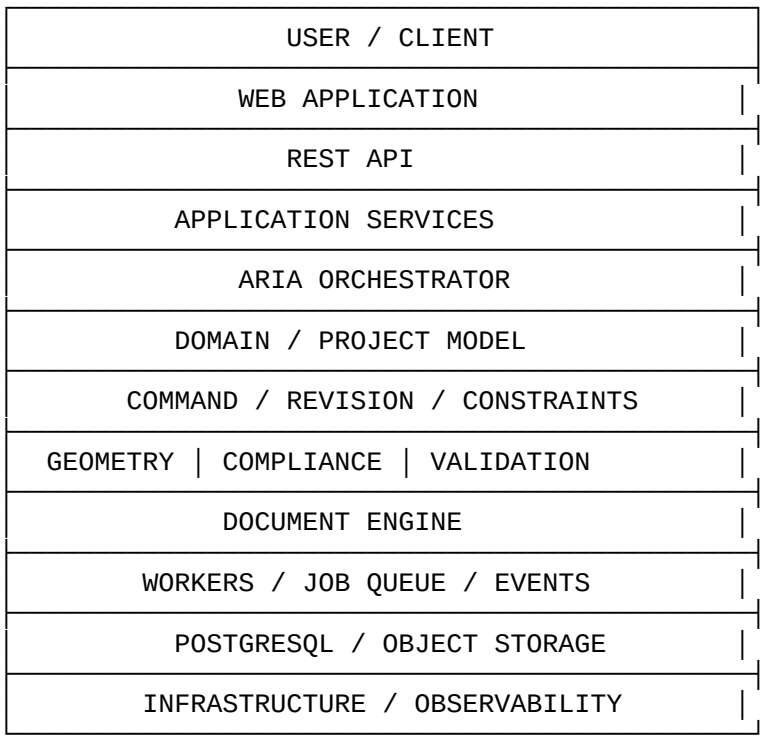
```
CREATE PROJECT
 ↓
CREATE MODEL
 ↓
COMMAND
 ↓
REVISION
 ↓
VALIDATION
 ↓
DRAWING
```

Once that loop is executable, every subsequent OneDraft capability can attach to the established computational foundation.

---

143. FINAL STACK DECISION

OneDraft shall therefore be implemented as a layered computational platform:



The architectural direction is therefore established:

OneDraft is a computational architectural documentation platform in which Aria operates as the controlled intelligence layer over a versioned Project Model, with deterministic geometry, jurisdiction-aware compliance, validation, documentation generation, revision control, provenance, and customer acceptance forming the executable system beneath the user interface.